

《修仙文字游戏后端系统》数据库设计说明书

(参考国家标准数据库设计文档规范)

版本: V1.0

项目名称: 修仙文字游戏后端系统 (MCS Backend)

目录

- 引言
- 数据库需求分析
- 数据库概念结构设计
- 数据库逻辑结构设计
- 数据库物理设计
- 数据完整性设计
- 数据安全设计
- 数据库优化设计
- 总结

1. 引言

1.1 编写目的

本文档用于说明《修仙文字游戏后端系统》的数据库设计方案, 包括:

- 数据结构设计
- ER 模型设计
- 数据表结构设计
- 主外键关系
- 数据安全设计
- 数据完整性设计

用于指导数据库开发与后期维护。

1.2 数据库设计目标

数据库设计目标如下：

- 支持玩家成长系统
 - 支持修仙事件记录
 - 支持功法管理
 - 支持战斗记录
 - 支持 AI 调用日志
 - 保证数据完整性
 - 提高查询效率
-

2. 数据库需求分析

2.1 数据存储需求

系统需要存储：

数据类型	描述
玩家数据	玩家基础属性
功法数据	玩家获得功法
事件记录	修仙事件日志
战斗记录	大比结果
AI 调用日志	AI 请求与返回
回退日志	fallback 信息

2.2 数据关系需求

主要关系：

- 一个玩家拥有多个功法
- 一个玩家拥有多个事件记录
- 一个玩家拥有多个战斗记录

- 一个 AI 调用对应多个回退记录

3. 数据库概念结构设计

3.1 ER 模型设计

系统 ER 模型如下：

Player

| 1

|

| N

GongFa

Player

| 1

|

| N

EventLog

Player

| 1

|

| N

BattleRecord

3.2 实体说明

Player（玩家实体）

用于保存玩家基本信息。

GongFa（功法实体）

用于保存功法数据。

EventLog（事件实体）

用于保存修仙事件。

BattleRecord（战斗实体）

用于保存天机大比记录。

AiCallLog（AI 日志实体）

用于记录 AI 调用。

4. 数据库逻辑结构设计

4.1 Player 玩家表

表名

player

表结构

字段名	类型	约束	描述
player_id	INT	PK	玩家 ID
player_name	VARCHAR(50)	NOT NULL	玩家名称
stage	INT	DEFAULT 1	境界
spirit	INT	DEFAULT 0	灵力
dao	INT	DEFAULT 0	道心
hp	INT	DEFAULT 100	气血
mp	INT	DEFAULT 100	灵力值
fame	INT	DEFAULT 0	名望
stone	INT	DEFAULT 0	灵石

4.2 GongFa 功法表

表结构

字段名	类型	约束	描述
gongfa_id	INT	PK	功法 ID
player_id	INT	FK	玩家 ID
name	VARCHAR(100)	NOT NULL	功法名称
rarity	VARCHAR(30)	NOT NULL	功法品阶
description	TEXT	NULL	功法描述
speed_bonus	FLOAT	DEFAULT 0	修炼加成

4.3 EventLog 事件表

字段名	类型	约束	描述
event_id	INT	PK	事件 ID
player_id	INT	FK	玩家 ID
scene	VARCHAR(100)	NULL	场景
action	VARCHAR(50)	NULL	行动
title	VARCHAR(100)	NULL	标题
desc	TEXT	NULL	描述
summary	TEXT	NULL	总结

4.4 BattleRecord 战斗记录表

字段名	类型	约束	描述
battle_id	INT	PK	战斗记录 ID
player_id	INT	FK	玩家 ID
enemy_name	VARCHAR(100)	NULL	对手名称

字段名	类型	约束	描述
result	VARCHAR(20)	NULL	战斗结果
reward_exp	INT	DEFAULT 0	修为奖励
reward_stone	INT	DEFAULT 0	灵石奖励

4.5 AiCallLog AI 调用日志表

字段名	类型	约束	描述
log_id	INT	PK	日志 ID
endpoint	VARCHAR(100)	NULL	接口路径
request_data	TEXT	NULL	请求数据
response_data	TEXT	NULL	返回数据
success	BOOLEAN	NULL	是否成功
error_message	TEXT	NULL	错误信息

5. 数据库物理设计

5.1 数据库选型

推荐使用 MySQL 8.0。

原因：

- 稳定性高
- 支持事务
- 社区完善
- 支持高并发

5.2 存储引擎

采用：

InnoDB

支持：

- 事务
- 外键
- 行级锁

5.3 索引设计

主键索引

表	主键
player	player_id
gongfa	gongfa_id
battle_record	battle_id

普通索引

字段	用途
player_name	玩家查询
rarity	功法筛选
enemy_name	战斗查询

6. 数据完整性设计

6.1 实体完整性

所有表均设置主键。

6.2 参照完整性

采用外键约束：

FOREIGN KEY(player_id)

REFERENCES player(player_id)

6.3 用户定义完整性

例如：

- 灵石不可为负数
 - 玩家等级必须大于 0
 - 功法品阶不能为空
-

7. 数据安全设计

7.1 权限控制

数据库账户分为：

用户	权限
admin	全部权限
app_user	增删改查
readonly	查询权限

7.2 数据备份

采用：

- 每日自动备份
 - 周期归档
 - 异地备份
-

8. 数据库优化设计

8.1 查询优化

采用：

- 索引优化

- 分页查询
 - 联表优化
-

8.2 高并发优化

后期可增加：

- Redis 缓存
 - 读写分离
 - 数据分片
-

9. 总结

本文档完成了《修仙文字游戏后端系统》的数据库设计。

数据库方案具备：

- 清晰的数据结构
- 合理的实体关系
- 完整的约束设计
- 良好的扩展能力
- 高可维护性

满足课程设计与后期项目扩展需求。